

Quasi-Randomized Networks for forecasting

thierrymoudiki.github.io

04/7/2019

Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 RVFL Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- 3 - Contributing

Plan

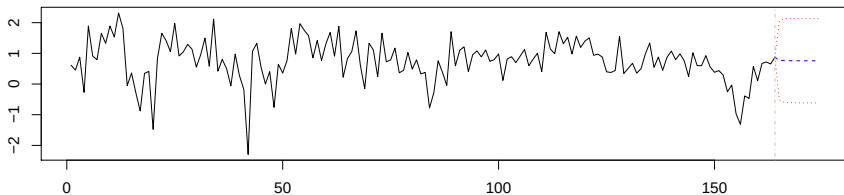
- **0 - Why?**
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 RVFL Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- 3 - Contributing

0 - Why?

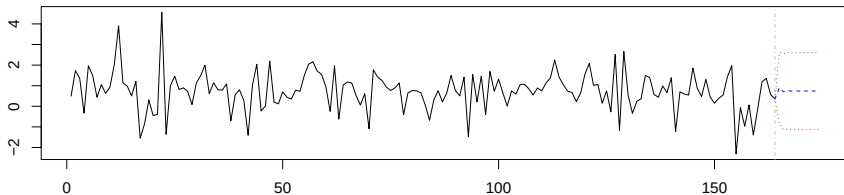
Why?!

0 - Why?

Forecast of series consumption



Forecast of series income



0 - Why?

```
## Granger causality test
##
## Model 1: consumption ~ Lags(consumption, 1:1) + Lags(income
## Model 2: consumption ~ Lags(consumption, 1:1)
##      Res.Df Df      F Pr(>F)
## 1      160
## 2      161 -1  1.8938 0.1707

## Granger causality test
##
## Model 1: income ~ Lags(income, 1:1) + Lags(consumption, 1:1)
## Model 2: income ~ Lags(income, 1:1)
##      Res.Df Df      F      Pr(>F)
## 1      160
## 2      161 -1  26.201 8.728e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Mostly based on:

- Moudiki, T., Planchet, F., & Cousin, A. (2018). Multiple Time Series Forecasting Using Quasi-Randomized Functional Link Neural Networks. *Risks*, 6, 22.
- **nnetsauce** Python package available at: github.com/thierrymoudiki/nnetsauce
- Want to **participate**? Extend the ressources? **Infinite possibilities.**

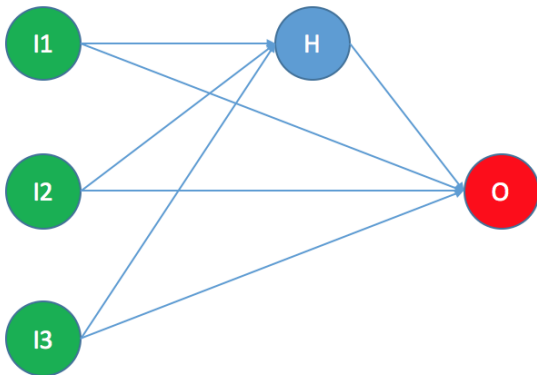
Plan

- 0 - Why?
- **1 - Random Vector Functional Link (RVFL) networks**
- **1 - 1 RVFL Description**
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 RVFL Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- 3 - Contributing

1 - 1 RVFL Description

$y \in \mathbb{R}^n$, to be explained by $\mathbf{x}^{(j)}$, $j \in \{1, \dots, p\}$

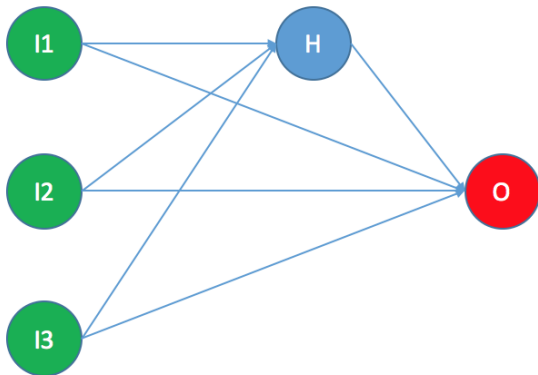
$$y = \beta_0 + \sum_{j=1}^p \beta_j \mathbf{x}^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p W^{(j,l)} \mathbf{x}^{(j)} \right) + \epsilon$$



1 - 1 RVFL Description

$y \in \mathbb{R}^n$, to be explained by $X^{(j)}$, $j \in \{1, \dots, p\}$

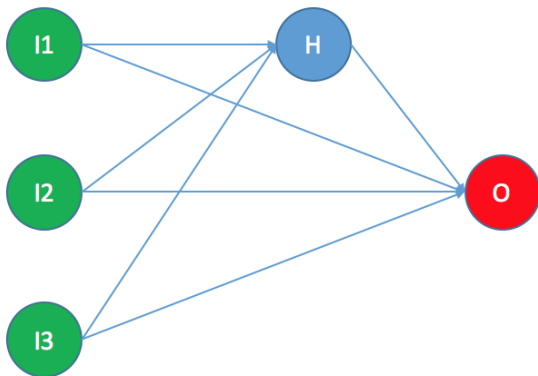
$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p w^{(j,l)} X^{(j)} \right) + \epsilon$$



1 - 1 RVFL Description

$y \in \mathbb{R}^n$, to be explained by $X^{(j)}$, $j \in \{1, \dots, p\}$

$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p W^{(j,l)} X^{(j)} \right) + \epsilon$$



1 - 1 RVFL Description

$y \in \mathbb{R}^n$, to be explained by $X^{(j)}, j \in \{1, \dots, p\}$

$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p W^{(j,l)} X^{(j)} \right) + \epsilon$$

With:

- g : *activation function* $\rightarrow$ **nonlinearity**
- L : number of **nodes** in the **hidden layer**
- $W^{(j,l)}$, hidden layer: **simulated** from uniform distribution
- β_j and γ_l : **learned** from the observed data y and $X^{(j)}, j \in \{1, \dots, p\}$
- ϵ : **residuals**
- optional **direct link**

1 - 1 RVFL Description

- **Result:** *RVFL networks are **universal approximators** of a continuous function on a bounded finite dimensional set [...].* From Igel and Pao (1995). *Stochastic choice of basis functions in adaptive function approximation and the functional-link net.*
- **Danger:** Overfitting
- **Overfitting?** Increasing the number nodes $L \rightarrow$ in-sample bias decreases $\rightarrow$ **out-of-sample error increases**
- $\Rightarrow$ **Regularization**

1 - 1 RVFL Description

Overfitting?

From “An Introduction to Statistical Learning” (Springer, 2013), authors: G. James, D. Witten, T. Hastie and R. Tibshirani

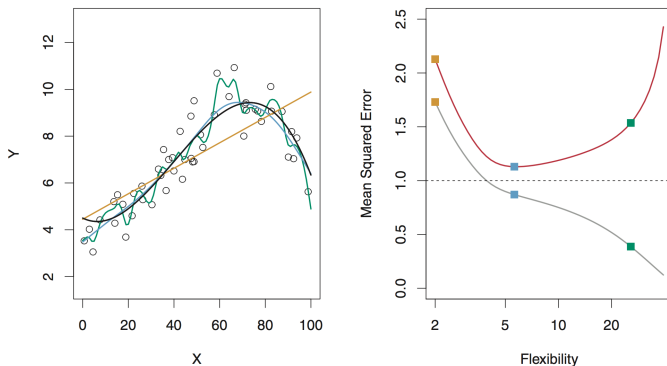


FIGURE 2.9. Left: Data simulated from f , shown in black. Three estimates of f are shown: the linear regression line (orange curve), and two smoothing spline fits (blue and green curves). Right: Training MSE (grey curve), test MSE (red

Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- **1 - 2 RVFL applied to Multivariate Time series (MTS)**
- 1 - 3 RVFL Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- 3 - Contributing

1 - 2 RVFL applied to Multivariate Time series (MTS)

- Time series $(X_t^{(j)})_{t,j=1,\dots,p}$, observed at $n \in \mathbb{N}^*$ discrete dates
- **aim**: forecasting the p time series at time $n+h$, $h \in \mathbb{N}^*$
- **output variables** for RVFL regression (**multiple outputs**):

$$Y^{(j)} = \left(X_n^{(j)}, \dots, X_{n+k}^{(j)} \right)^T$$

- $X_n^{(j)}$: most recent value of j^{th} time series
- output variables stored in **response**:

$$\mathbf{Y} \in \mathbb{R}^{(n-k) \times p}$$

- lags stored as **covariates** in:

$$\mathbf{X} \in \mathbb{R}^{(n-k) \times (k \times p)}$$

1 - 2 RVFL applied to Multivariate Time series (MTS)

- $\mathbf{X}$: p blocks of k lags, for each one of the observed p time series.
- j_0^{th} block of $\mathbf{X}$, for $j_0 \in \{1, \dots, p\}$ contains k columns:

$$\left(X_{n-i}^{(j_0)}, \dots, X_{k+1-i}^{(j_0)} \right)^T \quad (1)$$

with $i \in \{1, \dots, k\}$

- possibility to add other regressors, such as dummy variables, indicators of special events
- **Multitask learning:** **all** covariates shared by the p time series

1 - 2 RVFL applied to Multivariate Time series (MTS)

Illustration:

- $p = 2$ time series $X^{(1)}$ and $X^{(2)}$
- $n = 5$ dates $t_1 < \dots < t_5$,
- $X^{(2)} := (X_{t_1}^{(1)}, \dots, X_{t_5}^{(1)})$ and $X^{(2)} := (X_{t_1}^{(2)}, \dots, X_{t_5}^{(2)})$
- $k = 2$ lags
- $L = 3$ nodes in hidden layer.

Response variables:

$$\mathbf{Y} = \begin{pmatrix} X_{t_5}^{(1)} & X_{t_5}^{(2)} \\ X_{t_4}^{(1)} & X_{t_4}^{(2)} \\ X_{t_3}^{(1)} & X_{t_3}^{(2)} \end{pmatrix}$$

Covariates:

$$\mathbf{X} = \begin{pmatrix} X_{t_4}^{(1)} & X_{t_3}^{(1)} & X_{t_4}^{(2)} & X_{t_3}^{(2)} \\ X_{t_3}^{(1)} & X_{t_2}^{(1)} & X_{t_3}^{(2)} & X_{t_2}^{(2)} \\ X_{t_2}^{(1)} & X_{t_1}^{(1)} & X_{t_2}^{(2)} & X_{t_1}^{(2)} \end{pmatrix}$$

1 - 2 RVFL applied to Multivariate Time series (MTS)

Illustration:

- $p = 2$ time series $X^{(1)}$ and $X^{(2)}$
- $n = 5$ dates $t_1 < \dots < t_5$,
- $X^{(2)} := (X_{t_1}^{(1)}, \dots, X_{t_5}^{(1)})$ and $X^{(2)} := (X_{t_1}^{(2)}, \dots, X_{t_5}^{(2)})$
- $k = 2$ lags
- $L = 3$ nodes in hidden layer.

Coefficients in hidden layer:

$$\mathbf{W} = \begin{pmatrix} W^{(1,1)} & W^{(1,2)} & W^{(1,3)} \\ W^{(2,1)} & W^{(2,2)} & W^{(2,3)} \\ W^{(3,1)} & W^{(3,2)} & W^{(3,3)} \\ W^{(4,1)} & W^{(4,2)} & W^{(4,3)} \end{pmatrix}$$

1 - 2 RVFL applied to Multivariate Time series (MTS)

In the paper:

$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p \mathbf{w}^{(j,l)} X^{(j)} \right) + \epsilon$$

- **Hidden layer** elements $\mathbf{w}^{(j,l)}$: **low discrepancy** sequence
- **Discrepancy** of sequence of N points in $V \in [0, 1)^d$:

$$\sup_{V \in [0,1)^d} \left| \frac{\text{number of points in } V}{N} - v(V) \right|$$

where $v(V)$ = volume of V : **how well the points are dispersed within V ?**

1 - 2 RVFL applied to Multivariate Time series (MTS)

In the paper:

residuals ϵ :

- univariate time series: ARIMA, Exponential smoothing.

1 - 2 RVFL applied to Multivariate Time series (MTS)

In the paper:

$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p W^{(j,l)} X^{(j)} \right) + \epsilon$$

- **Constraints** on coefficients: $\|\beta\|^2 \leq u$ and $\|\gamma\|^2 \leq v \rightarrow$ reduce overfitting as **ridge regression**

1 - 2 RVFL applied to Multivariate Time series (MTS)

In the paper:

$$y = \beta_0 + \sum_{j=1}^p \beta_j X^{(j)} + \sum_{l=1}^L \gamma_l g \left(\sum_{j=1}^p W^{(j,l)} X^{(j)} \right) + \epsilon$$

- **Caution:** columns of X must be on the **same scale**
- For example:

$$X \rightarrow \frac{X - \mu}{\sigma}$$

or

$$X \rightarrow \frac{X - \min(X)}{\max(X) - \min(X)}$$

- Scaling on training set **must be applied to test set**

1 - 2 RVFL applied to Multivariate Time series (MTS)

```
# simulation taille et poids
set.seed(123)
n <- 4
height <- runif(n, min = 1.5, max = 1.9)
weight <- runif(n, min = 50, max = 90)
# standardisation
scaled_height <- scale(height)[,1]
scaled_weight <- scale(weight)[,1]
# g(XW)
w <- runif(2)
drop(tanh(cbind(height, weight)%*%w))
```

```
## [1] 1 1 1 1
```

```
drop(tanh(cbind(scaled_height, scaled_weight)%*%w))
```

```
## [1] -0.2054163 -0.2335470 -0.4062359 0.7051213
```


1 - 2 RVFL applied to Multivariate Time series (MTS)

Solving for Regression coefficients

- Lagrangian, one Lagrange multiplier λ_i per constraint:

$$\mathcal{L}(\mathbf{X}; \beta, \gamma) = ||y - \mathbf{X}\beta - \Phi(\mathbf{X})\gamma||^2 + \lambda_1 \beta^T \beta + \lambda_2 \gamma^T \gamma$$

with $\mathbf{X}$ = lagged predictors, $\Phi(\mathbf{X})$ = transformed lagged predictors

- 1st derivatives of $\mathcal{L}$ according to β and γ lead to:

$$\begin{pmatrix} \mathbf{X}^T \mathbf{X} + \lambda_1 I_{k \times p} & \mathbf{X}^T \Phi(\mathbf{X}) \\ \Phi(\mathbf{X})^T \mathbf{X} & \Phi(\mathbf{X})^T \Phi(\mathbf{X}) + \lambda_2 I_L \end{pmatrix} \begin{pmatrix} \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} \mathbf{X}^T y \\ \Phi(\mathbf{X})^T y \end{pmatrix}$$

1 - 2 RVFL applied to Multivariate Time series (MTS)

Solving for Regression coefficients

- We denote:

$$A = \begin{pmatrix} \mathbf{X}^T \mathbf{X} + \lambda_1 I_{k \times p} & \mathbf{X}^T \Phi(\mathbf{X}) \\ \Phi(\mathbf{X})^T \mathbf{X} & \Phi(\mathbf{X})^T \Phi(\mathbf{X}) + \lambda_2 I_L \end{pmatrix} =: \begin{pmatrix} B & C^T \\ C & D \end{pmatrix}$$

and $S = D - CB^+C^T$. Use algo for **blockwise matrix inversion**:

$$A^+ = \begin{pmatrix} B^+ + B^+C^TS^+CB^+ & -B^+C^TS^+ \\ -S^+CB^+ & S^+ \end{pmatrix} =: \begin{pmatrix} A_1^+ & A_2^+ \\ A_3^+ & A_4^+ \end{pmatrix}$$

S^+ and B^+ : Moore-Penrose pseudo-inverse of S and B .

1 - 2 RVFL applied to Multivariate Time series (MTS)

- Choosing λ_1 and λ_2 , # lags, L ? → **Rolling origin forecasting**

224

I. Kaastra, M. Boyd / Neurocomputing 10 (1996) 215–236

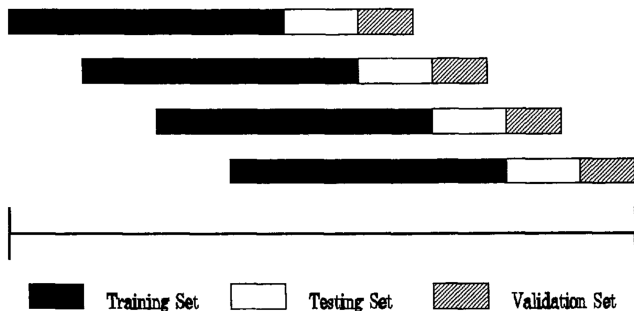


Fig. 3. Walk-forward sliding windows testing routine.

1 - 2 RVFL applied to Multivariate Time series (MTS)

- Choosing λ_1 and λ_2 , # lags, L ? $\rightarrow$ **Rolling origin forecasting**

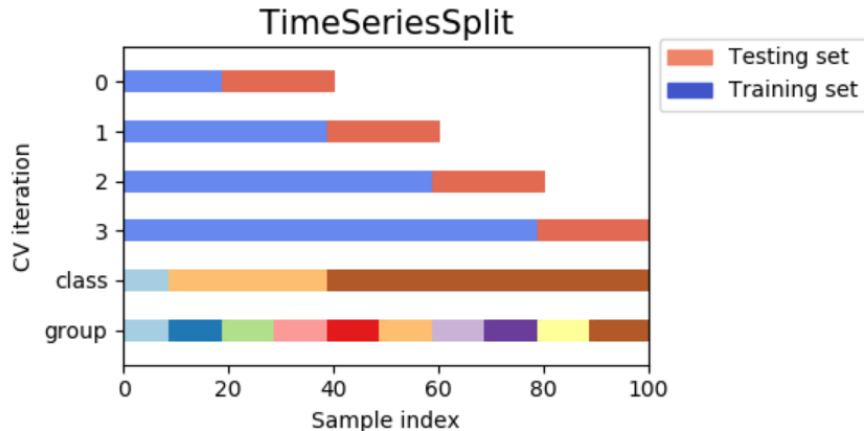
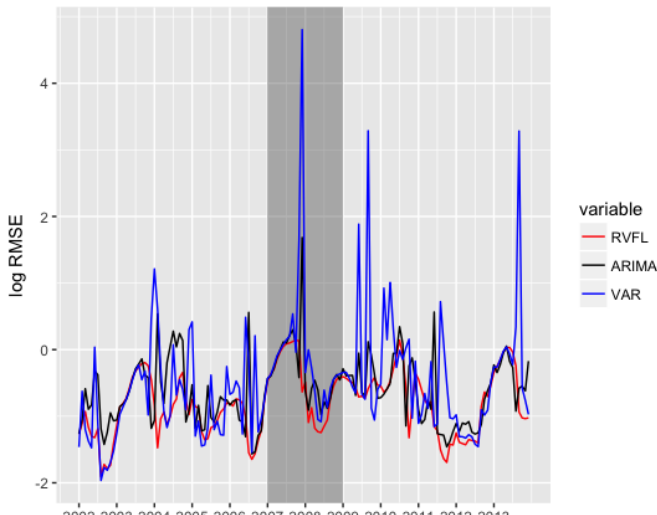


Figure 6: **Time series cross-validation** (source: scikit-learn.org)

1 - 2 RVFL applied to Multivariate Time series (MTS)

Results

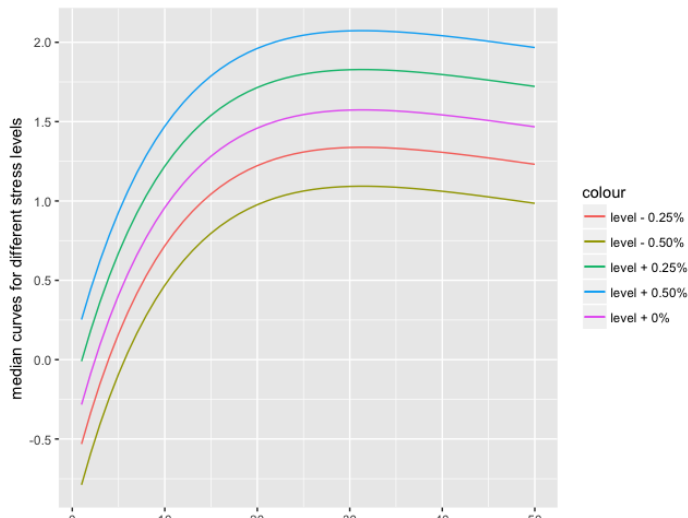
On forecasting interest rates (Nelson-Siegel/Diebold-Li model)



1 - 2 RVFL applied to Multivariate Time series (MTS)

Results

On stress-testing forecast curves



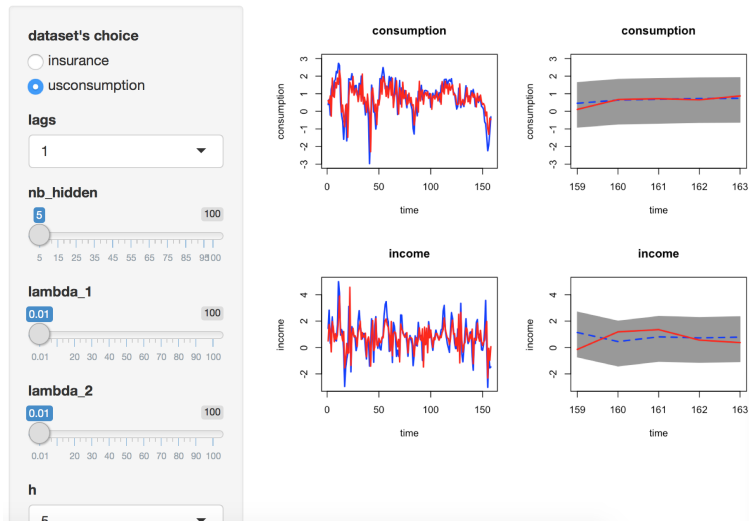
Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 **RVFL Examples and demo**
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- 3 - Contributing

1 - 3 RVFL Examples and demo

Demo

Example of forecasts with ridge2



Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
 - 1 - 1 RVFL Description
 - 1 - 2 RVFL applied to Multivariate Time series (MTS)
 - 1 - 3 Examples and demo
- **2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)**
 - **2 - 1 Installing [nnetsauce](#)**
 - 2 - 2 What's in the `nnetsauce`?
 - 2 - 3 Example
 - 3 - Contributing

2 - 1 Installing nnetsauce

- nnetsauce? **Generalize the principle of RVFL** + others
 - Python package under the BSD license available at:
- ① Github: github.com/thierrymoudiki/nnetsauce
 - ② Command line: `> pip install nnetsauce`

Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- **2 - 2 What's in the nnetsauce?**
- 2 - 3 Example
- 3 - Contributing

2 - 2 What's in the nnetsauce?

For time series:

- Generalization of RVFL
- Recurrent Networks (RNN)

2 - 2 What's in the nnetsauce?

For time series:

- **Generalization of RVFL**
- Recurrent Networks (RNN)

2 - 2 What's in the nnetsauce?

Generalization of RVFL:

Still

- $p = 2$ time series $X^{(1)}$ and $X^{(2)}$
- $n = 5$ dates $t_1 < \dots < t_5$,
- $X^{(2)} := (X_{t_1}^{(1)}, \dots, X_{t_5}^{(1)})$ and $X^{(2)} := (X_{t_1}^{(2)}, \dots, X_{t_5}^{(2)})$
- $k = 2$ lags, $L = 3$ nodes.

Response variables:

$$\mathbf{Y} = \begin{pmatrix} X_{t_5}^{(1)} & X_{t_5}^{(2)} \\ X_{t_4}^{(1)} & X_{t_4}^{(2)} \\ X_{t_3}^{(1)} & X_{t_3}^{(2)} \end{pmatrix}$$

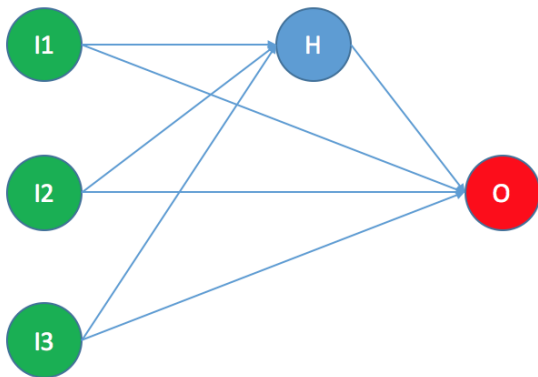
Covariates:

$$\mathbf{X} = \begin{pmatrix} X_{t_4}^{(1)} & X_{t_3}^{(1)} & X_{t_4}^{(2)} & X_{t_3}^{(2)} \\ X_{t_3}^{(1)} & X_{t_2}^{(1)} & X_{t_3}^{(2)} & X_{t_2}^{(2)} \\ X_{t_2}^{(1)} & X_{t_1}^{(1)} & X_{t_2}^{(2)} & X_{t_1}^{(2)} \end{pmatrix}$$

2 - 2 What's in the nnetsauce?

Generalization of RVFL: With one layer

$$y = f(\mathbf{X}^{(j)}, g(W^{(j,l)} \mathbf{X}^{(j)})) ; f = \text{your choice}$$



Inputs

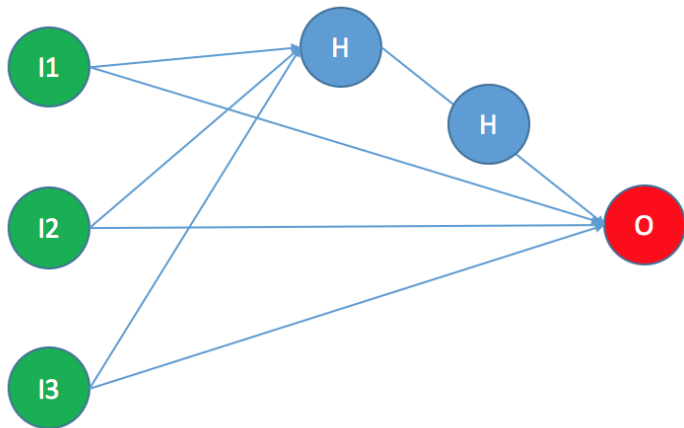
Hidden

Outputs

2 - 2 What's in the nnetsauce?

Generalization of RVFL: Can go deeper

$$y = f(\mathbf{x}^{(j)}, g(\dots, g(W^{(j,l)} \mathbf{x}^{(j)}))) ; f = \text{your choice}$$



2 - 2 What's in the nnetsauce?

Generalization of RVFL:

$$y = f(X^{(j)}, g(\dots, g(W^{(j,l)} X^{(j)}))) ; f = \text{your choice}$$

- f = linear regression, random forest, regression trees, xgboost etc.
- f , **Bayesian** model (Gaussian Process, Bayesian Ridge Regression e.g)?
 $\Rightarrow$ nnetsauce can obtain **credible intervals** around prediction $\hat{y}$

2 - 2 What's in the nnetsauce?

For time series:

- Generalization of RVFL
- **Recurrent Networks (RNN)**

2 - 2 What's in the nnetsauce?

Recurrent Networks (RNN)

Series:

$$(x_t)_{t \geq 0} \quad (2)$$

State variable:

$$h_t = g([x_t; h_{t-1}], \mathbf{W}) \quad (3)$$

Output:

$$o_t = f(h_t) \quad (4)$$

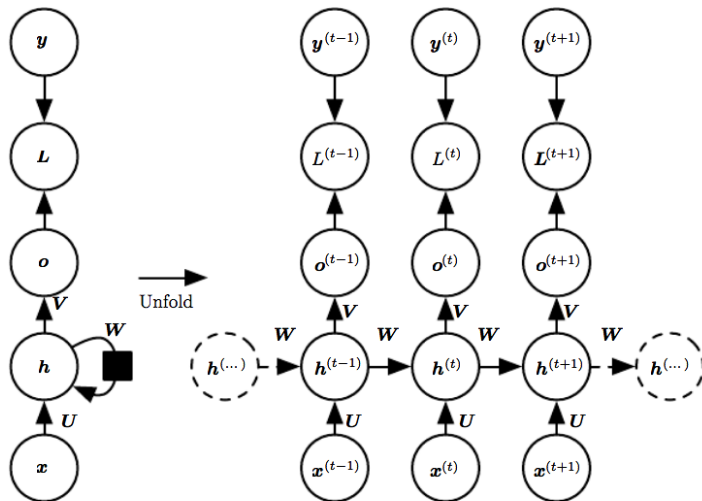
Loss:

$$L(y_t, o_t) \quad (5)$$

2 - 2 What's in the nnetsauce?

Recurrent Networks (RNN)

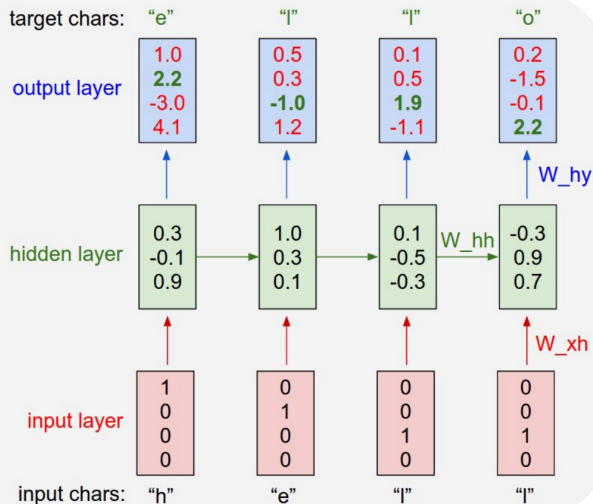
(From 'Deep Learning', Goodfellow et. al (2016))



2 - 2 What's in the nnetsauce?

Recurrent Networks (RNN)

(From karpathy.github.io (2015))



2 - 2 What's in the nnetsauce?

Recurrent Networks (RNN) in nnetsauce

- With other links (between output and target) **than logistic**
- With **exponential smoothing**

2 - 2 What's in the nnetsauce?

Recurrent Networks (RNN) in nnetsauce

Series:

$$(x_t)_{t \geq 0}$$

State variable:

$$h_t = g(\alpha \mathbf{W} h_{t-1} + (1 - \alpha) \mathbf{U} x_t + b)$$

$\mathbf{W}$ and $\mathbf{U}$ quasi – random

Output:

$$o_t = f(h_t)$$

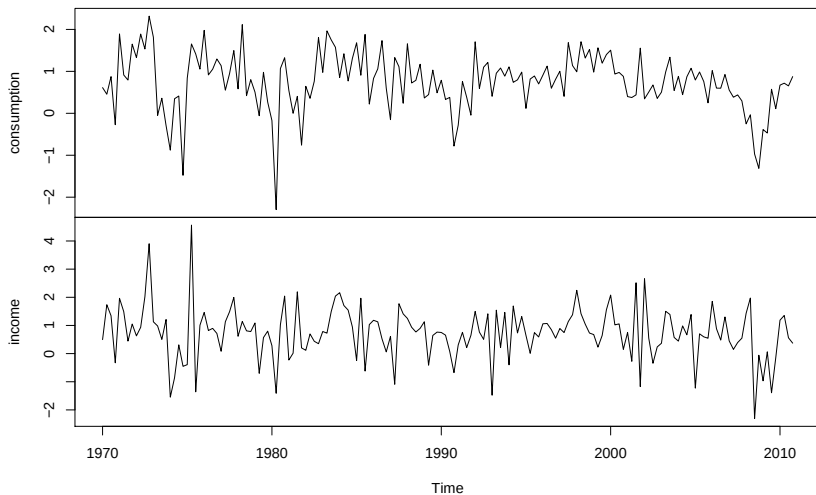
f = your choice of classifier/regressor

Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- **2 - 3 Example**
- 3 - Contributing

2 - 3 Example

fpp::usconsumption



2 - 3 Example

```
summary(fpp::usconsumption)
```

##	consumption	income
##	Min. :-2.2966	Min. :-2.3086
##	1st Qu.: 0.4050	1st Qu.: 0.3099
##	Median : 0.8005	Median : 0.7480
##	Mean : 0.7553	Mean : 0.7366
##	3rd Qu.: 1.1141	3rd Qu.: 1.2622
##	Max. : 2.3171	Max. : 4.5642

2 - 3 Example

- With one layer

```
import numpy as np
import nnetsauce as ns
from sklearn import linear_model, gaussian_process

data_path = "/Users/moudiki/Documents/Presentations/Presentation_mbour_sfds
usconsumption = np.loadtxt(data_path + "usconsumption.csv", delimiter=";")

# 1 - One layer -----

f1 = linear_model.BayesianRidge()

# create object
obj_MTS = ns.MTS(f1, lags = 2, n_hidden_features=25,
                 direct_link=True, bias=True,
                 nodes_sim='sobel', activation_name='relu')

# fit 'nnetsauce' object to 'usconsumption' data
obj_MTS.fit(usconsumption)

# predict
print(obj_MTS.predict())

# predict with standard dev.
print(obj_MTS.predict(return_std=True)[2:4])
```

2 - 3 Example

- With stacked layers

```
0
1 # 2 - deep model -----
2
3 layer1_f = gaussian_process.GaussianProcessRegressor()
4
5 layer2_f = ns.CustomRegressor(obj = layer1_f, n_hidden_features=5,
6                               direct_link=True, bias=True,
7                               dropout = 0.5,
8                               nodes_sim='sobol', activation_name='tanh',
9                               n_clusters=2)
10
11 layer3_f = ns.CustomRegressor(obj = layer2_f, n_hidden_features=5,
12                               direct_link=True, bias=True,
13                               dropout = 0.5,
14                               nodes_sim='hammersley', activation_name='sigmoid',
15                               n_clusters=2)
16
17 deep_obj_MTS = ns.MTS(layer3_f, lags = 1, n_hidden_features=5,
18                       direct_link=True, bias=True,
19                       dropout = 0.5,
20                       nodes_sim='sobol', activation_name='relu')
21
22 # fit 'nnetsauce' object to 'usconsumption' data
23 deep_obj_MTS.fit(usconsumption)
24
25 # predict
26 print(deep_obj_MTS.predict())
27
28 # predict with standard dev.
29 print(deep_obj_MTS.predict(return_std=True)[2:4])
30
31
```

2 - 3 Example

- With RNNs

```
2 # 3 - RNN
3
4 n, p = usconsumption.shape
5 inputs = usconsumption[:-1,:]
6 targets = usconsumption[1:n,:]
7
8 rnn7 = ns.RNNRegressor(obj=f1, bias=True, direct_link=True,
9                        nodes_sim='sobol', activation_name='relu',
10                       alpha = 0.9, n_hidden_features = 20,
11                       dropout=0.1, n_clusters=0)
12 rnn7.fit(inputs = inputs, targets = targets)
13 rnn7.predict(return_std=True)
```

Figure 13: RNN

Plan

- 0 - Why?
- 1 - Random Vector Functional Link (RVFL) networks
- 1 - 1 RVFL Description
- 1 - 2 RVFL applied to Multivariate Time series (MTS)
- 1 - 3 Examples and demo
- 2 - More Quasi-Randomized Networks forecasting with [nnetsauce](#)
- 2 - 1 Installing [nnetsauce](#)
- 2 - 2 What's in the `nnetsauce`?
- 2 - 3 Example
- **3 - Contributing**

3 - Contributing

- **Git/Github:** greaaaaat for online collaboration.
- ___Installing Git:___
- <https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>
- **Git 101:**
<https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>
- **Github 101:**
- <https://guides.github.com/activities/hello-world/>

Thanks

Thank you for your attention.